**Name : Pushpak Bajanghate**

**Roll no : C1_14**

**NLP Practical 8**

**Movie Sentiment Analyzer**

**Github : https://github.com/PushpakBajanghate/Movie_Sentiment_Analyzer**

Aim : Write the python code to perform sentiment analysis using NLP Instructions for Preparing the Movie Review Sentiment Analysis PDF Document.

DataSet Used : IMDB DataSet.csv

Code :

```
import pandas as pd

df = pd.read_csv("IMDB Dataset.csv", engine='python', on_bad_lines='skip')

import pandas as pd

import numpy as np

import nltk

import re

import matplotlib.pyplot as plt

import seaborn as sns

import pickle


from nltk.corpus import stopwords

from nltk.tokenize import word_tokenize


from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.model_selection import train_test_split
```

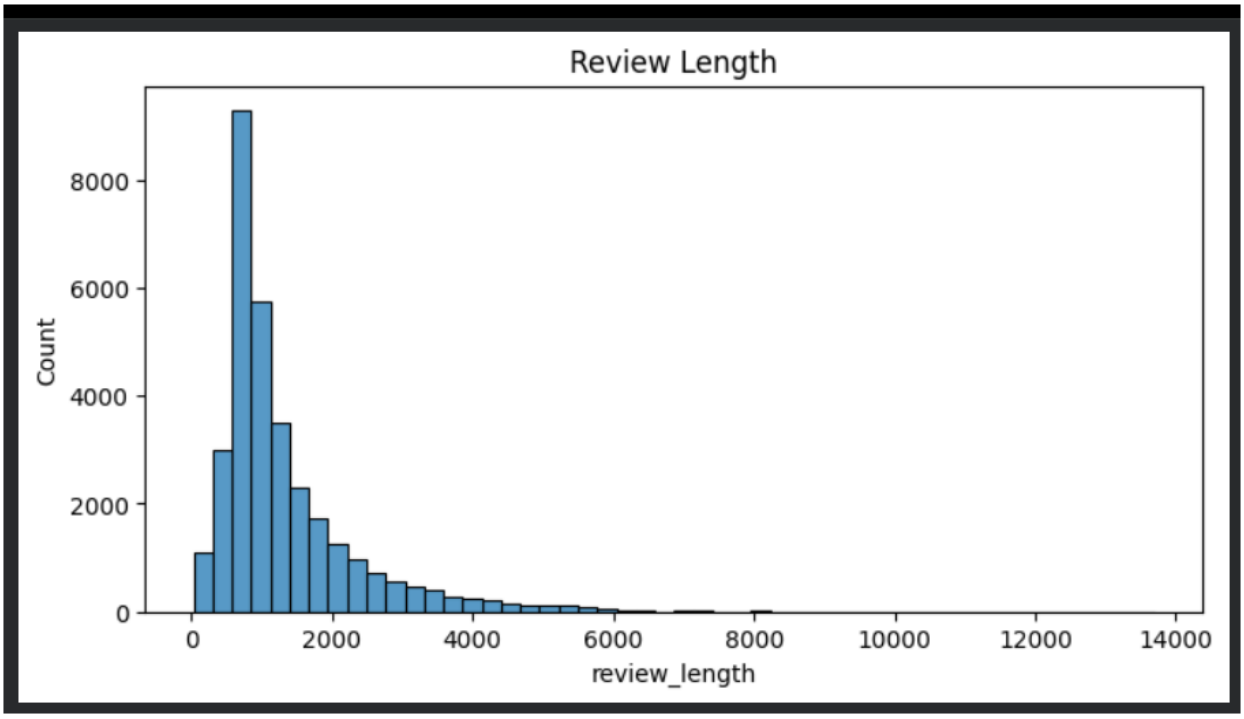
```
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score


from sklearn.naive_bayes import MultinomialNB

from sklearn.linear_model import LogisticRegression

from sklearn.svm import LinearSVC

from sklearn.ensemble import RandomForestClassifier


from wordcloud import WordCloud


import spacy

import gradio as gr

nltk.download('punkt')

nltk.download('stopwords')

nltk.download('punkt_tab')

df = pd.read_csv("/content/IMDB Dataset.csv", engine='python', on_bad_lines='skip')


df.head()
```

| | review | sentiment |
|---|---|---|
| 0 | One of the other reviewers has mentioned that ... | positive |
| 1 | A wonderful little production. <br /><br />The... | positive |
| 2 | I thought this was a wonderful way to spend ti... | positive |
| 3 | Basically there's a family where a little boy ... | negative |
| 4 | Petter Mattei's "Love in the Time of Money" is... | positive |

Next steps: Generate code with df  New interactive sheet

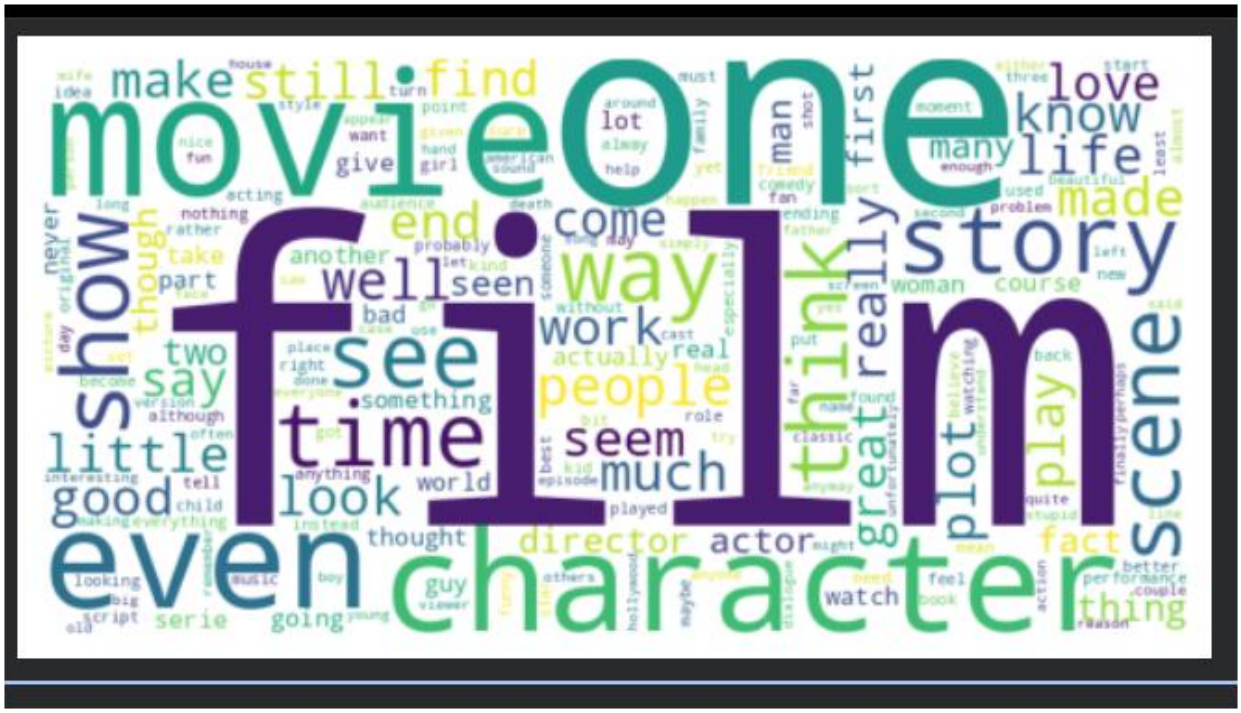
```
print(df.columns)


text_col = df.columns[0]

target_col = df.columns[1]


df = df[[text_col, target_col]]

df.columns = ["review", "sentiment"]

df.shape

df.info()

sns.countplot(x="sentiment", data=df)

plt.title("Sentiment Distribution")

plt.show()

df["review_length"] = df["review"].apply(len)
```

```
plt.figure(figsize=(8,4))

sns.histplot(df["review_length"], bins=50)

plt.title("Review Length")

plt.show()
```




```
print(df.isnull().sum())


print("Duplicates:", df.duplicated().sum())


df.dropna(inplace=True)

df.drop_duplicates(inplace=True)

from nltk.stem import WordNetLemmatizer

nltk.download('wordnet')


lemmatizer = WordNetLemmatizer()
```

```
stop_words = set(stopwords.words("english"))

def preprocess(text):

    text = str(text).lower()
    text = re.sub(r"<.*?>"," ",text)
    text = re.sub(r"http\S+"," ",text)
    text = re.sub(r"[^a-zA-Z]"," ",text)

    tokens = word_tokenize(text)
    tokens = [w for w in tokens if w not in stop_words and len(w)>2]

    tokens = [lemmatizer.lemmatize(w) for w in tokens]

    return " ".join(tokens)

df["clean_text"] = df["review"].apply(preprocess)
text = " ".join(df["clean_text"])

wordcloud = WordCloud(width=800,height=400,background_color="white").generate(text)

plt.imshow(wordcloud)
plt.axis("off")
plt.show()
```


film
one
movie
character
even
time
story
scene
see
way
people
little
show
good
look
work
seem
much
think
make
still
find
love
know
life
made
come
end
well
seen
say
two
play
plot
great
really
first
man
many
lot
give
actor
director
fact
thing
watch
guy
going
serie
thought
actually
real
something
another
bad
woman
course
though
never
take
part
nothing
rather
acting
world
anything
tell
child
looking
music
boy
script
old
interesting
making everything
instead
played
episode
best
kid
hollywood
maybe
others
feel
book
action
reason
better
performance
couple
stupid
quite
finally perhaps
back
watching
understand
believe
classic
unfortunately
found
far
put
screen
yet
second
problem
beautiful
used
enough
fun
nice
house
wife
idea
turn
point
style
appear
hand
girl
sound
audience
death
fan
ending
father
comedy
friend
happen
part
left
new
said
case
use
place
right
done
version
although
often
become
everyone
get
head
cast
without
especially
someone
simply
long
day
kind
let
probably
rather
nothing
acting
original
day
since

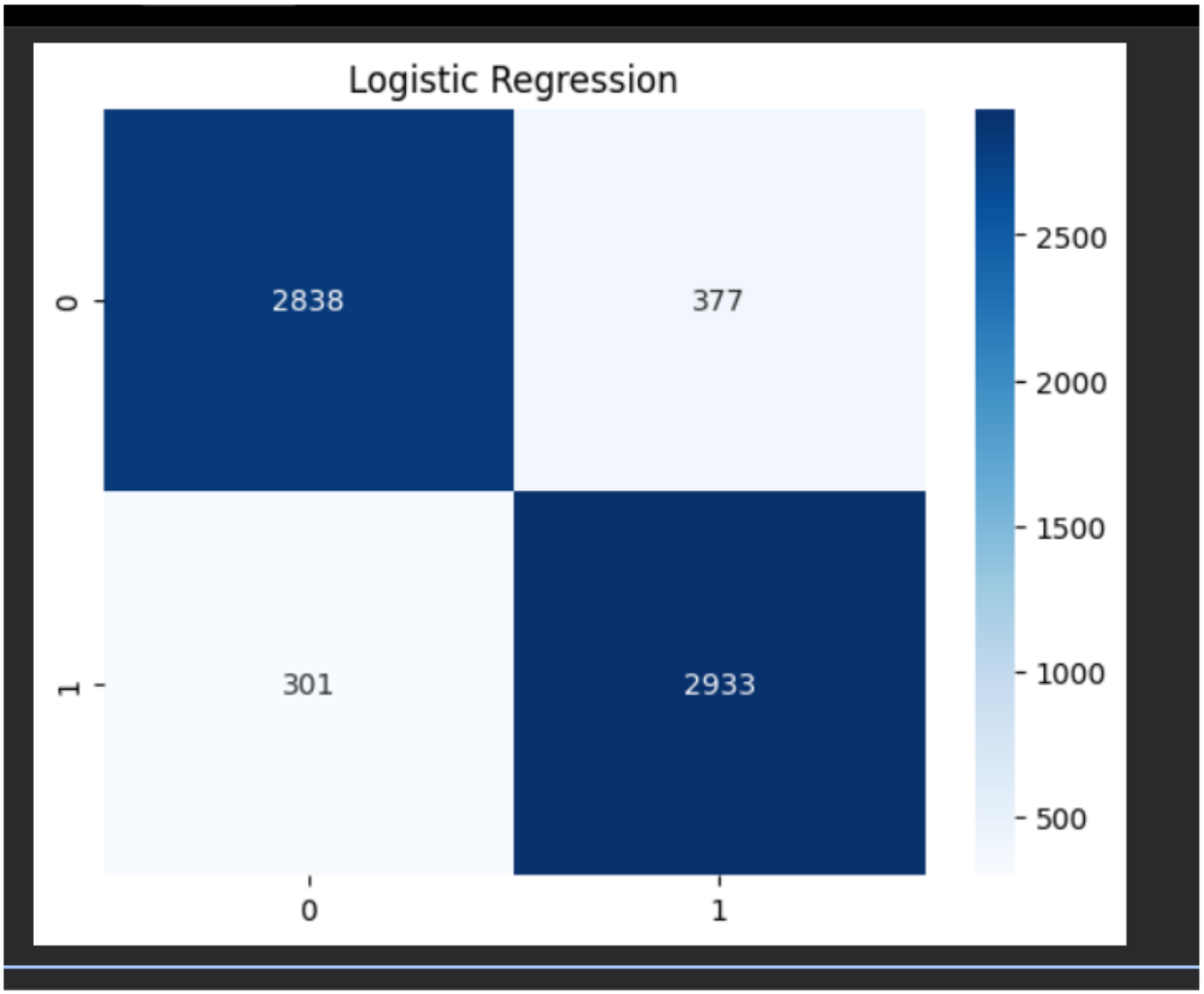
```
max_df=0.9
)

X = vectorizer.fit_transform(df["clean_text"])
y = df["sentiment"]
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y
)

vectorizer = TfidfVectorizer(
    max_features=25000,
    ngram_range=(1,2),
    min_df=2,
    max_df=0.9
)

X = vectorizer.fit_transform(df["clean_text"])
y = df["sentiment"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
models = {
"Naive Bayes": MultinomialNB(),
"Logistic Regression": LogisticRegression(max_iter=2000),
```

```
"Linear SVC": LinearSVC(C=1.8),
"Random Forest": RandomForestClassifier(n_estimators=400)
}
results = []

for name, model in models.items():

    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    results.append([
        name,
        accuracy_score(y_test, pred),
        precision_score(y_test, pred),
        recall_score(y_test, pred),
        f1_score(y_test, pred)
    ])

results_df = pd.DataFrame(
results,
columns=["Model","Accuracy","Precision","Recall","F1 Score"]
)

results_df
```

| | Model | Accuracy | Precision | Recall | F1 Score |
|---|---|---|---|---|---|
| 0 | Naive Bayes | 0.880609 | 0.872960 | 0.891903 | 0.882330 |
| 1 | Logistic Regression | 0.900272 | 0.890674 | 0.913402 | 0.901895 |
| 2 | Linear SVC | 0.892004 | 0.890132 | 0.895318 | 0.892718 |
| 3 | Random Forest | 0.864273 | 0.859434 | 0.872212 | 0.865776 |

Next steps: Generate code with results_df New interactive sheet

```
best_model_name = results_df.loc[results_df["Accuracy"].idxmax(),"Model"]

best_model = models[best_model_name]

best_model.fit(X_train, y_train)

pred = best_model.predict(X_test)

cm = confusion_matrix(y_test, pred)

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title(best_model_name)
plt.show()
```

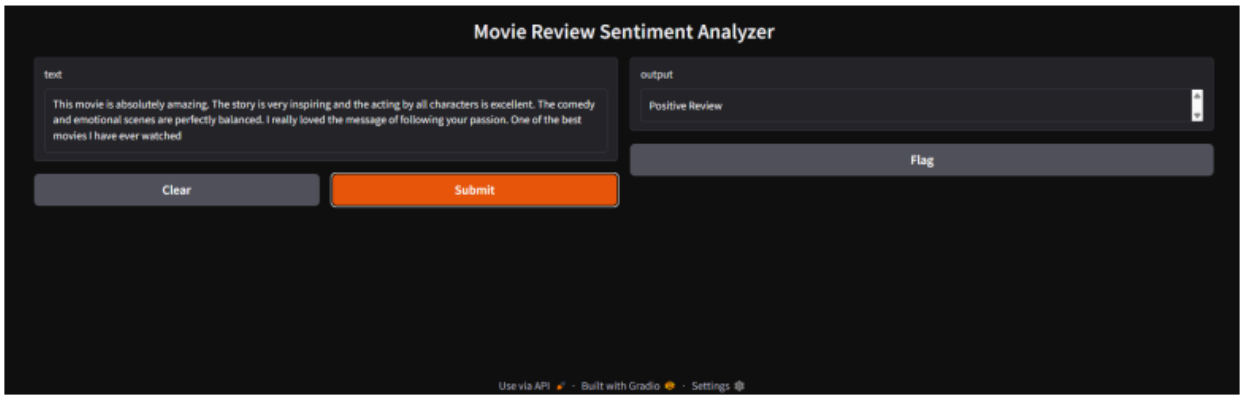


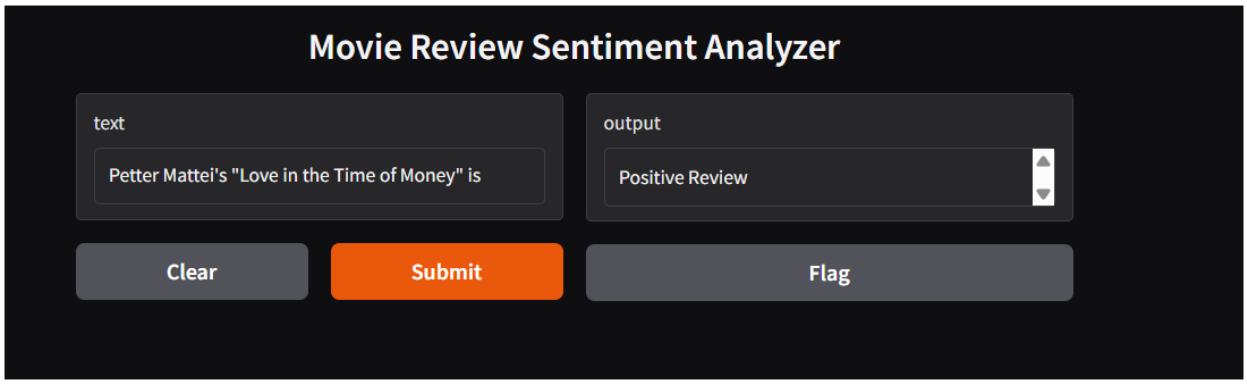

pickle.dump(best_model, open("sentiment_model.pkl","wb")) pickle.dump(vectorizer, open("vectorizer.pkl","wb"))

def predict(text): clean = preprocess(text) vec = vectorizer.transform([clean]) return best_model.predict(vec)[0] predict("This movie is absolutely amazing")

def predict_sentiment(text): result = predict(text) return "Positive Review" if result==1 else "Negative Review" interface = gr.Interface( fn=predict_sentiment, inputs="text", outputs="text", title="Movie Review Sentiment Analyzer" ) interface.launch()

Output :




Best Performance Model : Logistic Regression

Accuracy: 90.02%